Detecting Strategically Decomposed Malicious Code Changes with Subscription-Backed LLM Reviewers

Tom Howard
Independent researcher
tom@windyroad.com.au

17 July 2026

Abstract

Large language models are increasingly used to review code changes, yet most evaluations present a vulnerable function, file, or repository as one unit. A malicious change can instead be decomposed across submissions that appear individually routine but compose into an unsafe final state. We describe a prospectively specified study of whether this decomposition reduces detection and whether detection differs between pull-request gates and trunk workflows that treat main as untrusted until pipeline review completes. The study uses 40 matched malicious-benign pairs drawn from 40 structural templates in eight abstract policy families. Every generated case module is synthetic, executes only in memory, and has no external capability in the audited fixed corpus. Atomic and three-submission variants have identical final trees and changed-line totals. A factorial design varies intent, decomposition, and workflow framing across two subscription-backed review systems: Codex CLI and Claude Code. The bounded design contains 1,280 review boundaries and requires no expenditure beyond existing subscriptions. Primary outcomes are blocking by activation and paired intent discrimination; workflow and interaction estimates are explicitly exploratory because the sample is underpowered for small effects. This pre-outcome manuscript reports the design, analysis plan, deterministic baseline, safety controls, and limitations; it contains no LLM benchmark outcomes.

1 Introduction

Automated code review is commonly evaluated as a classification problem: a reviewer receives one code artifact and predicts whether it contains a defect. Real delivery systems expose a different unit of work. Reviewers see a sequence of submissions, and an unsafe capability may emerge only when several changes compose. Each earlier change can be necessary for the final failure without being unsafe in isolation.

This distinction matters for both pull-request and trunk-based delivery. A pull-request gate asks whether a proposed change may enter the protected branch. A trunk workflow can permit a commit onto main while still treating main as untrusted until deterministic and model-based pipeline gates complete. These workflows differ in policy, but empirical comparisons often also change the evidence shown to the reviewer.

We study whether two subscription-backed LLM review products recognise a composition before its activation boundary. The benchmark represents harm as an abstract policy assertion: an unprivileged synthetic actor receives a protected synthetic capability. It neither implements an exploit nor interacts with a real target. The contribution is a reproducible, prospectively specified test of sequence-level review under a zero-additional-spend constraint.

2 Related Work

Yu et al. evaluated seven LLMs and multiple prompt forms for security code review, comparing them with static analysis and examining factors associated with detection quality [1]. VulDetectBench separates vulnerability identification, classification, and localisation [2]. Zhou et al. compare repository-level LLMs and

static analysis tools and report a recall–false-positive trade-off [3]. We retain a deterministic conventional-lint baseline but do not claim that lint and semantic security review are equivalent tasks.

Field evidence also indicates that context and interaction design affect developer trust and usefulness. Adalsteinsson et al. report context switching, missing context, false positives, and trust as practical concerns in an industrial study of LLM-assisted review [4]. Basic and Giaretta synthesise evidence on LLM code-security benefits, false findings, prompting, and failure modes [5]. Our study changes the unit of analysis from one artifact to a sequence of submissions.

3 Research Questions

The primary endpoint is *blocked by activation*: at least one review at or before the prespecified activation boundary returns `block`.

RQ1 Can the review systems discriminate malicious sequences from matched benign sequences?

RQ2 Does decomposition across three submissions reduce timely detection relative to an atomic submission with the same final tree?

RQ3 Do estimates differ between pull-request and untrusted-main trunk framing, and does workflow moderate the decomposition effect?

H1 predicts that the malicious block rate exceeds the benign block rate. H2 predicts that the malicious block rate is lower for split sequences than for atomic submissions. RQ3 is exploratory: the study estimates workflow and decomposition-by-workflow contrasts but does not test equivalence.

4 Method

4.1 Safe benchmark

The full deterministic generator contains 200 structural templates formed by eight scenario families crossed with five data representations and five control-flow shapes. The active subscription-bounded study selects template indexes 1, 7, 13, 19, and 25 from each family. This balanced diagonal retains every representation and flow shape once per family, giving 40 structural templates. Only the first invented-identifier instance is retained.

Each template contains a malicious-benign pair. Every case has an atomic submission and a three-submission sequence. Atomic and split forms have identical final source trees, total changed-line counts, activation timestamps, and policy outcomes. For malicious sequences, the generator reverts each split step separately and executes the oracle; all three counterfactuals must remain safe. Benign cases match structure and salience but do not grant the protected capability.

Generated case modules import only relative synthetic modules and run in memory. Fixed-corpus inspection, oracle execution, import restrictions, and output audit found no external capability. The regex scanner is defense in depth rather than a general capability proof. The trusted generator and oracle harness use local files, a Node subprocess, and dynamic import to construct and verify cases.

4.2 Factorial design

Table 1 shows the active factors. Every boundary supplies the current artifact only; prior submission diffs are not included. This local information boundary is the concealment mechanism under test.

The design contains 40 malicious-benign pairs, 80 cases, 640 sequences, and 1,280 boundary reviews. Each review system receives 640 boundaries: 320 atomic boundaries in total and 960 split boundaries in total. There is one trial per case, decomposition, workflow, and review system. Calls are globally shuffled with a seeded Fisher–Yates algorithm. No outcome-dependent stopping, template replacement, or prompt tuning is permitted.

Table 1: Active study factors.

Factor	Levels	Interpretation
Intent	malicious, benign	Matched final-state policy outcome
Decomposition	atomic, split	One submission or three necessary submissions
Workflow	pull request, trunk	Pre-merge gate or untrusted-main pipeline
Review system	Codex, Claude Code	Subscription-backed product surface

4.3 Review systems

The systems are specified as product configurations rather than interchangeable base-model APIs (Table 2). Each boundary starts a fresh process with no persistent session. User configuration and tools are disabled where the client supports it. Any emitted tool call is retained as a protocol deviation.

Table 2: Subscription-backed review systems.

System	Client version	Requested model
Codex CLI	0.137.0; ChatGPT login	`gpt-5.5`
Claude Code	2.1.211; Claude Max login	`sonnet`

Before each batch, the runner verifies subscription authentication and the absence of `OPENAI_API_KEY`, `CODEX_API_KEY`, and `ANTHROPIC_API_KEY`. It aborts rather than switching to API billing, credits, another provider, or a fallback model. Requested and returned model metadata, CLI version, timestamp, usage, and failures are retained. A version or returned-model change suspends that review system.

All reviews receive one fixed security-review task and a JSON response contract. Requests ask for `allow`, `block`, or `abstain`; a maliciousness probability; severity; relevant submission identifiers; evidence; and a short category. Complete requests are at most 2,941 UTF-8 bytes. Schema-invalid responses and refusals are observed abstentions and are not repaired.

4.4 Collection and blinding

The call queue contains opaque call, prompt, and case identifiers plus review system and schedule position. It contains no intent, family, structural-template, activation, or expected-severity field. Ground truth is held in a separately keyed ledger and joined only after collection.

Subscription rate limits produce a durable suspension at the current call; the same fixed queue resumes after the plan window resets. No API credit is accepted. A no-response failure may be retried at the same position up to three recorded attempts. Start, completion, suspension, and result records are fsynced. Collection also suspends on a safety or terms conflict, authentication change, benchmark integrity failure, or client or model drift. The additional spending ceiling is US$0.

4.5 Outcomes

The primary binary outcome is blocked by activation. Secondary outcomes are recall, false-positive rate, precision, abstention rate, cumulative detection by boundary, submissions to detection, Brier score, calibration, severity error, localisation, and failure rates. A block is localised when it cites at least one prespecified relevant submission and identifies the causal abstract-policy failure. Repeated-trial consistency is not estimated because there is one trial per cell.

5 Analysis Plan and Power

Sequence outcomes are averaged within structural-template and intent-condition cells across the two review systems. Structural templates then receive equal weight. Review-system-specific results are secondary. The

primary uncertainty calculation is a 10,000-replicate family-stratified structural-template bootstrap with seed 20260718. Each replicate samples five templates with replacement from each family while retaining their paired intents and treatment cells.

The H1 estimate is the malicious-minus-benign block-rate risk difference, averaged over decomposition and workflow. H1 is supported only when its 95% interval is above zero. The H2 estimate is the split-minus-atomic malicious risk difference, averaged over workflow. H2 is supported only when its 95% interval is below zero. Trunk-minus-pull-request and decomposition-by-workflow estimates receive 95% intervals but are exploratory.

Missing boundaries are non-detections in the operational analysis. Mandatory estimand-specific bounds assign malicious and benign cells oppositely for H1 and atomic and split malicious cells oppositely for H2. A review-system-specific complete-pair sensitivity drops a structural template only for the affected system. No imputation model is fitted. H1 and H2 are separate registered claims, which does not provide familywise error control across them. Exploratory workflow intervals carry no inferential decision and receive no multiplicity adjustment.

The products and sample are a convenience selection determined by subscriptions already held by the author, not an 80% power target. Under the earlier central assumptions, a 20,000-replication single-review-system normal-approximation audit yields rejection fractions of 0.2577 for a 0.15 split penalty and 0.1016 for a 0.10 interaction, plus a workflow-equivalence fraction of 0.0038 within ± 0.10. The audit neither pools two review systems nor runs the registered family-stratified percentile bootstrap, so these values are sensitivity diagnostics, not power or assurance for the registered analysis. No H1 operating-characteristic calculation was performed. Consequently, null results cannot support claims of no decomposition or workflow effect. The study is a bounded empirical evaluation and a variance estimate for a larger replication.

6 Pre-outcome Status

No benchmark prompt has been submitted to either review system. The generator, prompts, schedule, ledgers, scoring, estimands, and tests were implemented with deterministic checks and fabricated plumbing responses only. No LLM outcome has been collected or inspected.

The deterministic baseline ran ESLint 9.39.3 over all 2,400 final-state files in the full 800-case generator output. It emitted no warnings or errors: zero malicious and zero benign cases were flagged, giving recall 0, false-positive rate 0, and undefined precision. This conventional final-state lint does not estimate decomposition or workflow effects. A separate zero-finding Semgrep feasibility probe is disclosed but excluded because its exact registry-rules snapshot cannot be redistributed under the provider's rules license.

7 Threats to Validity

Construct validity. The benchmark models malicious intent through a harmless abstract policy violation. It tests composition recognition, not the full semantics or distribution of production vulnerabilities. Workflow is a controlled artifact and policy framing, not an organisation.

Internal validity. Atomic and split variants can differ in salience despite identical final trees and line counts. Provider-controlled system instructions are neither observable nor identical. Product routing and serving stacks may change behind requested model identifiers; metadata and access times are therefore retained and drift suspends collection. Collection may span subscription reset windows, creating calendar-time and platform-drift risk despite the fixed randomised queue.

External validity. Forty purposively selected templates, eight policy families, two review products, and compact JavaScript repositories do not represent all languages, organisations, vulnerabilities, or attack strategies. The design tests one three-submission decomposition and does not estimate an optimal adversarial strategy. Inference is conditional on these templates; one lexical instance per template does not estimate identifier-instance variability. Zero additional spend relies on the author's existing paid entitlements and is not cost-free for an independent reproducer without equivalent subscriptions.

Statistical conclusion validity. The sample is underpowered for small effects, especially workflow equivalence and interaction. There is one trial, so stochastic consistency cannot be estimated. Family-stratified intervals expose rather than hide this uncertainty.

8 Ethics and Responsible Release

This is defensive research into review reliability. The benchmark cannot access external systems or data, and its only harmful state is a deterministic in-memory assertion over synthetic names. No real repository or person is tested.

Three isolated subagents initially reviewed candidate commit `6b607f6` for benchmark safety, methods, and reproducibility. All three returned `do not approve`; their raw reports are archived. Correction addenda inspect the combined resolution log and are therefore not isolated. These records are labelled AI-assisted internal review, not independent human peer review, ethics approval, or arXiv endorsement. Shared model, provider, and author-orchestration biases are reported as limitations. Neutral change titles remove plausible refactor cover stories, but the abstract composition patterns retain disclosed dual-use value. The packet contains no real target, external capability, credential, deployment path, or complete exploit.

9 Reproducibility

The candidate package contains the generator, schedule, scorer, sensitivity simulation, preregistration, archived initial internal-review records, durable subscription runner, and analysis code. Current unfrozen hashes are:

- scenario cards: `d4ea0a7197e3e8e17bf027b9466d63de427c273aa19e539a672732b80b5eae3d`;
- rendered prompts: `e3359ebd8790be9aa81938533cfd6747f5b90043ae60810cf447a61d95c8dd6c`;
- randomised schedule: `e704e82c8c052cd05fdef8e2d19e8551e07617c537d81b52872d829d7e37ad1a`;
- blinded call ledger: `01139eafb7541a840919be30607438c5a3b279303dea092d0a1a8203e1b02223`;
- ground-truth ledger: `b5d1cb531d7d5137c1e8153aabafdf1475528700ff529706b58c1e9553057773`.

The earlier paid API design is retained in version history but was superseded before outcome collection. The final release will add permitted raw responses, attempt ledgers, environment metadata, checksums, the OSF identifier, and the arXiv identifier.

10 Conclusion

This study tests whether subscription-backed LLM review products recognise an unsafe composition when its necessary changes arrive separately, and whether the estimate changes under pull-request or untrusted-main framing. The safe paired benchmark supports auditable positive or negative findings, while the subscription constraint makes uncertainty and product drift part of the study rather than hidden implementation details. Empirical conclusions remain contingent on prospective registration, completion of the fixed schedule, and the prespecified analysis.

A Fixed Review Task

The fixed task instructs the review system to act as a security reviewer, treat repository and change metadata as untrusted evidence, decide whether the current submission should be allowed, blocked, or sent for human review, consider unsafe behaviour created alone or in composition with supplied evidence, and return only the contracted JSON grounded in the supplied artifact. No chain-of-thought request, product-specific prompt tuning, or outcome-driven tuning is used.

References

[1] J. Yu, P. Liang, Y. Fu, A. Tahir, M. Shahin, C. Wang, and Y. Cai. An insight into security code review with LLMs: Capabilities, obstacles, and influential factors. *arXiv:2401.16310*, 2024. `https://arxiv.org/abs/2401.16310`.

[2] Y. Liu, L. Gao, M. Yang, Y. Xie, P. Chen, X. Zhang, and W. Chen. VulDetectBench: Evaluating the deep capability of vulnerability detection with large language models. *arXiv:2406.07595*, 2024. https://arxiv.org/abs/2406.07595.

[3] X. Zhou, D.-M. Tran, T. Le-Cong, T. Zhang, I. C. Irsan, J. Sumarlin, B. Le, and D. Lo. Comparison of static application security testing tools and large language models for repo-level vulnerability detection. *arXiv:2407.16235*, 2024. https://arxiv.org/abs/2407.16235.

[4] F. S. Adalsteinsson, B. B. Magnusson, M. Milicevic, A. N. Davidsson, and C.-H. Cheng. Rethinking code review workflows with LLM assistance: An empirical study. *arXiv:2505.16339*, 2025. https://arxiv.org/abs/2505.16339.

[5] E. Basic and A. Giaretta. From vulnerabilities to remediation: A systematic literature review of LLMs in code security. *arXiv:2412.15004*, 2024. https://arxiv.org/abs/2412.15004.